Centrul de Excelență în Informatică și Tehnologii Informaționale

Catedra Informatică I

Lucrare de studiu individual Nr.1-2

1. **Disciplina**: **Programarea vizuală-C#**

Tema:**Aplicație vizuală cu utilizarea controllerelor de bază**

Realizat : Malai Mihai

Verificat: Covali Eugenia

Nota_____

Chişinău 2026

Cuprins

Cuprins 1
1. Introducere 3
2. Partea I – Aplicație vizuală cu utilizarea controllerelor de bază 4
2.1. Plasarea componentelor pe suprafața ferestrei 4
2.2. Descrierea controllerelor utilizate 5
2.3. Elementele caracteristice ale componentelor: proprietăți, metode, evenimente 5
2.3.1. Button – butoanele de comandă 5
2.3.2. DataGridView – grila de afișare 6
2.3.3. RadioButton – selecție rol 7
2.3.4. MenuStrip și ToolStripMenuItem 8
2.4. Modificarea proprietăților 9
3. Partea II – Aplicație vizuală complexă cu implementarea controllerelor frecvent utilizate 11
3.1. Componente frecvent utilizate și descrierea lor 11
3.1.1. Butoane de comandă (Button) 11
3.1.2. Casete de editare (TextBox) 11
3.1.3. Etichete (Label) 12
3.1.4. Cutii de editare multilinie (RichTextBox) 12
3.1.5. Meniuri (MenuStrip, ToolStripMenuItem, ToolStripSeparator) 13
3.1.6. Grupuri de butoane (RadioButton în Panel) 13
3.1.7. Cutii de grupare (GroupBox) 14
3.2. Adăugarea, excluderea și modificarea ferestrelor (MDI) 14
3.3. Modificarea stării și comportamentul componentelor conform specificațiilor 15
3.4. Conversia șirurilor de caractere 16
4. Partea III – Aplicație vizuală cu utilizarea controllerelor pentru lucrul cu bazele de date 18
4.1. Stabilirea conexiunilor cu bazele de date 18
4.2. Verificarea conexiunilor cu bazele de date 18
4.3. Securizarea accesului la bazele de date 19
4.4. Implementarea operațiilor CRUD 20
4.5. Implementarea interogărilor de selecție, modificare, inserare și excludere 22
4.6. Crearea și afișarea rapoartelor 23
5. Concluzie 24
6. Bibliografie 25
7. Anexe 26
Anexa 1 – Codul sursă complet al aplicației CompanyITApp 26

1. Introducere

Prezenta lucrare de studiu individual are ca scop familiarizarea cu mediul de dezvoltare vizuală C# Windows Forms și cu utilizarea controllerelor de bază în crearea aplicațiilor desktop. Lucrarea descrie proiectarea, implementarea și testarea unei aplicații funcționale complete pentru gestionarea angajaților dintr-o companie IT, realizată în limbajul C# cu ajutorul framework-ului .NET și al tehnologiei Windows Forms.

Aplicația CompanyITApp oferă un set complet de funcționalități: adăugarea, modificarea, ștergerea și căutarea angajaților, afișarea statisticilor (număr total, salariu mediu), exportul datelor în format CSV, și utilizarea arhitecturii MDI (Multiple Document Interface) cu ferestre copil. Interfața este proiectată cu atenție la detalii vizuale: gradient de fundal, butoane cu colțuri rotunjite, stilizare DataGridView cu rânduri alternante și antet colorat.

Obiectivele principale ale lucrării sunt: (1) înțelegerea și utilizarea componentelor vizuale de bază din Windows Forms; (2) gestionarea evenimentelor și a stării controllerelor; (3) validarea și conversia datelor introduse de utilizator; (4) implementarea operațiilor CRUD pe o colecție de obiecte; (5) exportul datelor și afișarea rapoartelor statistice.

2. Partea I – Aplicație vizuală cu utilizarea controllerelor de bază

2.1. Plasarea componentelor pe suprafața ferestrei

Interfața principală a aplicației este realizată prin clasa Form1, care moștenește clasa de bază System.Windows.Forms.Form. Constructorul formularului configurează proprietățile generale ale ferestrei și apelează metodele de inițializare în ordinea corectă:

```
public Form1()
{
    this.Text = "\u2728 Companie IT - Gestionare Angaja\u021bi \u2728";
    this.Size = new Size(1050, 700);
    this.StartPosition = FormStartPosition.CenterScreen;
    this.IsMdiContainer = true;
    this.BackColor = Color.White;
    this.Paint += Form1_Paint;

    InitializeMenu();
    InitializeControls();
    LoadSampleData();
    ApplyFilter();
    UpdateStatistics();
}
```

Listing 1 – Constructorul clasei Form1

Componentele sunt plasate utilizând coordonate absolute (proprietatea Location de tip Point) și dimensiuni fixe (proprietatea Size de tip Size), respectând o structură logică în două zone principale:

- Zona stângă (x: 25–445px): GroupBox cu câmpurile de introducere a datelor angajatului, organizate printr-un TableLayoutPanel în 2 coloane.
- Zona dreaptă (x: 460–1000px): DataGridView pentru afișarea listei angajaților, câmpul de căutare și butonul de filtrare.
- Zona inferioară: butoanele de comandă (Adaugă, Modifică, Șterge, Curăță) la y=360px, și StatusStrip ancorat la baza ferestrei.

Plasarea se realizează programatic în metoda InitializeControls(). Fiecare componentă primește o locație și dimensiune specifică, iar la final este adăugată pe formular prin this.Controls.Add(). Ordinea adăugării controllerelor este importantă pentru Z-order (ordinea de suprapunere vizuală).

2.2. Descrierea controllerelor utilizate

Aplicația utilizează un set bogat de controllere Windows Forms, fiecare cu un rol bine definit în interfața utilizatorului. Mai jos sunt prezentate toate componentele folosite:

Proprietate / Element	Valoare / Descriere din aplicație
MenuStrip	Bara de meniu principală cu 3 meniuri: Fișier, Editare, Ajutor. BackColor dark (#323246), ForeColor alb.
ToolStripMenuItem	Elementele din meniuri: Exportă CSV, Ieșire, Curăță câmpuri, Despre. Fiecare are eveniment Click propriu.
GroupBox	employeeGroupBox – grupează câmpurile formularului. Size(420,300), font bold Segoe UI 11pt, ForeColor DarkSlateBlue.
TableLayoutPanel	tableLayout – 2 coloane (100px fix + 100% fill), 4 rânduri AutoSize. Asigură alinierea etichetă|câmp.
Label	nameLabel, roleLabel, salaryLabel, descLabel – etichete descriptive, TextAlign MiddleRight, Segoe UI 10pt Bold.
TextBox	nameTextBox (Nume), salaryTextBox (Salariu), searchTextBox (Căutare) – font Segoe UI 10pt, BorderStyle.FixedSingle.
RichTextBox	descriptionRichTextBox – câmp multilinie pentru Descriere, Height=60px, Segoe UI 10pt.
RadioButton	developerRadio, testerRadio, managerRadio – selecție exclusivă rol. Plasate în rolePanel cu locații fixe.
Panel	rolePanel – container pentru radio buttons, Height=40, AutoSize=true.
Button	addButton (verde), editButton (albastru), deleteButton (roșu), clearButton (portocaliu), searchButton (steel blue).
DataGridView	employeesDataGridView – afișare tabelară angajați. Size(540,400), FullRowSelect, ReadOnly, fără header rânduri.
StatusStrip	Bara de stare cu ToolStripStatusLabel. BackColor #34495E, afișează statistici în timp real.
ToolStripStatusLabel	statusLabel – Spring=true (se extinde), TextAlign MiddleLeft, text dinamic cu total angajați și salariu mediu.

2.3. Elementele caracteristice ale componentelor: proprietăți, metode, evenimente

Fiecare componentă vizuală expune trei categorii principale de elemente. Mai jos sunt prezentate detaliat pentru componentele principale din aplicație:

2.3.1. Button – butoanele de comandă

```
// Creare buton stilizat cu proprietăți personalizate:
private Button CreateStyledButton(string text, int x, int y, Color backColor)
```

```
{
    Button btn = new Button()
    {
        Text = text,
        Location = new Point(x, y),
        Size = new Size(95, 40),
        FlatStyle = FlatStyle.Flat,      // Proprietate: stil plat
        BackColor = backColor,           // Proprietate: culoare fundal
        ForeColor = Color.White,         // Proprietate: culoare text
        Font = new Font("Segoe UI", 9, FontStyle.Bold), // Proprietate: font
        Cursor = Cursors.Hand            // Proprietate: cursor mână la hover
    };
    btn.FlatAppearance.BorderSize = 0;   // Proprietate: fără bordură

    // Eveniment Paint: colțuri rotunjite cu GraphicsPath
    btn.Paint += (sender, e) => {
        GraphicsPath path = new GraphicsPath();
        int radius = 15;
        Rectangle rect = new Rectangle(0, 0, btn.Width - 1, btn.Height - 1);
        path.AddArc(rect.X, rect.Y, radius, radius, 180, 90);
        path.AddArc(rect.X + rect.Width - radius, rect.Y, radius, radius, 270, 90);
        path.AddArc(rect.X + rect.Width - radius,
            rect.Y + rect.Height - radius, radius, radius, 0, 90);
        path.AddArc(rect.X, rect.Y + rect.Height - radius, radius, radius, 90, 90);
        path.CloseFigure();
        btn.Region = new Region(path);   // Proprietate: formă buton
    };

    // Evenimente MouseEnter/MouseLeave: efect hover
    btn.MouseEnter += (sender, e) =>
        btn.BackColor = ControlPaint.Light(backColor, 0.7f);
    btn.MouseLeave += (sender, e) =>
        btn.BackColor = backColor;
    return btn;
}
```

Listing 2 – Metoda CreateStyledButton() cu proprietăți, metode și evenimente

2.3.2. DataGridView – grila de afișare

```
employeesDataGridView = new DataGridView()
{
    Location = new Point(460, 50),
    Size = new Size(540, 400),
    AllowUserToAddRows = false,          // Proprietate: fără rând nou de la user
    AllowUserToDeleteRows = false,       // Proprietate: user nu poate șterge
    ReadOnly = true,                     // Proprietate: doar citire
    SelectionMode = DataGridViewSelectionMode.FullRowSelect, // Selecție rând întreg
    MultiSelect = false,                 // Proprietate: o singură selecție
    BackgroundColor = Color.White,
```

```
    BorderStyle = BorderStyle.None,
    GridColor = Color.LightGray,
    RowHeadersVisible = false,          // Proprietate: fără header rânduri
    Font = new Font("Segoe UI", 10)
};

// Stilizare header coloane
employeesDataGridView.EnableHeadersVisualStyles = false;
employeesDataGridView.ColumnHeadersDefaultCellStyle.BackColor =
    Color.FromArgb(52, 73, 94);         // Proprietate stil: fundal header dark
employeesDataGridView.ColumnHeadersDefaultCellStyle.ForeColor = Color.White;
employeesDataGridView.ColumnHeadersDefaultCellStyle.Font =
    new Font("Segoe UI", 10, FontStyle.Bold);

// Rânduri alternante pentru lizibilitate
employeesDataGridView.AlternatingRowsDefaultCellStyle.BackColor =
    Color.FromArgb(240, 248, 255);

// Eveniment: selecția unui rând
employeesDataGridView.SelectionChanged += DataGridView_SelectionChanged;

// Metodă: redimensionare automată coloane
employeesDataGridView.AutoResizeColumns();
```

Listing 3 – Configurarea DataGridView cu proprietăți, metode și evenimente

2.3.3. RadioButton – selecție rol

```
// Creare și plasare RadioButton-uri în Panel
rolePanel = new Panel() { Height = 40, AutoSize = true };

developerRadio = new RadioButton()
{
    Text = "\U0001f468\u200d\U0001f4bb Developer", // Proprietate: text afișat
    AutoSize = true,                    // Proprietate: dimensiune automată
    Location = new Point(0, 5),         // Proprietate: locație în panel
    Checked = false                     // Proprietate: stare inițială
};
testerRadio = new RadioButton()
    { Text = "\U0001f9ea Tester", AutoSize = true, Location = new Point(110, 5) };
managerRadio = new RadioButton()
    { Text = "\U0001f4ca Manager", AutoSize = true, Location = new Point(220, 5) };

rolePanel.Controls.AddRange(            // Metodă: adăugare în panel
    new Control[] { developerRadio, testerRadio, managerRadio });

// Metodă helper pentru citirea rolului selectat:
private string GetSelectedRole()
{
    if (developerRadio.Checked) return "Developer"; // Proprietate: Checked
```

```
    if (testerRadio.Checked)   return "Tester";
    if (managerRadio.Checked)  return "Manager";
    return "";
}
```

Listing 4 – RadioButton: proprietăți și metode

2.3.4. MenuStrip și ToolStripMenuItem

```
private void InitializeMenu()
{
    mainMenu = new MenuStrip();
    mainMenu.BackColor = Color.FromArgb(50, 50, 70); // Proprietate: fundal dark
    mainMenu.ForeColor = Color.White;              // Proprietate: text alb
    mainMenu.Font = new Font("Segoe UI", 10, FontStyle.Bold);

    // Creare meniuri principale
    fileMenu = new ToolStripMenuItem("\U0001f4c1 Fi\u015fier");
    editMenu = new ToolStripMenuItem("\u270f\ufe0f Editare");
    helpMenu = new ToolStripMenuItem("\u2753 Ajutor");

    // Creare elemente de meniu
    exportMenuItem = new ToolStripMenuItem("\U0001f4be Export\u0103 \u00een CSV");
    exitMenuItem   = new ToolStripMenuItem("\U0001f6aa Ie\u015fire");
    clearMenuItem  = new ToolStripMenuItem("\U0001f9f9 Cur\u0103\u021b\u0103 c\u00e2mpuri");
    aboutMenuItem  = new ToolStripMenuItem("\u2139\ufe0f Despre");

    // Evenimente Click pe fiecare element
    exitMenuItem.Click += (s, e) => Application.Exit();
    clearMenuItem.Click += (s, e) => ClearInputFields();
    aboutMenuItem.Click += (s, e) => MessageBox.Show(
        "Aplica\u021bie modern\u0103 pentru gestionarea angaja\u021bilor IT",
        "Despre");
    exportMenuItem.Click += ExportToCsv;

    // Adăugare elemente în meniuri
    fileMenu.DropDownItems.Add(exportMenuItem);
    fileMenu.DropDownItems.Add(new ToolStripSeparator());
    fileMenu.DropDownItems.Add(exitMenuItem);
    editMenu.DropDownItems.Add(clearMenuItem);
    helpMenu.DropDownItems.Add(aboutMenuItem);

    mainMenu.Items.Add(fileMenu);
    mainMenu.Items.Add(editMenu);
    mainMenu.Items.Add(helpMenu);

    this.Controls.Add(mainMenu);
    this.MainMenuStrip = mainMenu; // Metodă: setare meniu principal al formularului
}
```

Listing 5 – Configurarea MenuStrip și a elementelor de meniu

2.4. Modificarea proprietăților

Proprietățile componentelor sunt modificate atât la inițializare (static), cât și dinamic în timpul execuției programului. Există două categorii de modificări:

a) Modificări statice – la inițializare:

Culorile, fonturile, dimensiunile și pozițiile sunt setate o singură dată în InitializeMenu() și InitializeControls(). De exemplu, butoanele editButton și deleteButton sunt dezactivate inițial:

```
editButton.Enabled = false;    // Proprietate Enabled = false la inițializare
deleteButton.Enabled = false;  // Activate doar când e selectat un angajat
```

Listing 6 – Dezactivare butoane la inițializare

b) Modificări dinamice – în timpul execuției:

Starea controllerelor se schimbă în funcție de acțiunile utilizatorului. La selectarea unui angajat în DataGridView, butoanele se activează și câmpurile se populează automat:

```
private void DataGridView_SelectionChanged(object sender, EventArgs e)
{
   if (employeesDataGridView.SelectedRows.Count > 0)
   {
      // Citire obiect legat de rândul selectat
      currentEmployee =
         (Employee)employeesDataGridView.SelectedRows[0].DataBoundItem;

      if (currentEmployee != null)
      {
         // Populare câmpuri cu datele angajatului selectat
         nameTextBox.Text = currentEmployee.Name;       // Modificare proprietate Text
         salaryTextBox.Text = currentEmployee.Salary.ToString();
         descriptionRichTextBox.Text = currentEmployee.Description;

         // Selectare radio button corespunzător rolului
         switch (currentEmployee.Role)
         {
            case "Developer": developerRadio.Checked = true; break;
            case "Tester":    testerRadio.Checked = true; break;
            case "Manager":   managerRadio.Checked = true; break;
         }

         // Activare butoane – modificare proprietate Enabled
         editButton.Enabled = true;
```

```
            deleteButton.Enabled = true;
        }
    }
    else
    {
        editButton.Enabled = false;   // Dezactivare când nimic selectat
        deleteButton.Enabled = false;
    }
}
```

Listing 7 – Modificarea dinamică a proprietăților la SelectionChanged

Un alt exemplu de modificare dinamică a proprietăților este efectul de hover pe butoane, implementat prin evenimentele MouseEnter și MouseLeave care modifică proprietatea BackColor:

```
// Efect hover: modificare proprietate BackColor la hover
btn.MouseEnter += (sender, e) =>
    btn.BackColor = ControlPaint.Light(backColor, 0.7f); // Mai deschis la hover
btn.MouseLeave += (sender, e) =>
    btn.BackColor = backColor;                    // Revine la original
```

Listing 8 – Modificarea proprietății BackColor prin evenimentele hover

Fundalul formularului este redesenat la fiecare redimensionare prin evenimentul Paint, utilizând un gradient liniar:

```
private void Form1_Paint(object sender, PaintEventArgs e)
{
    // Modificare vizuală dinamică: gradient de fundal
    using (LinearGradientBrush brush = new LinearGradientBrush(
        this.ClientRectangle,
        Color.FromArgb(240, 248, 255),   // Albastru-alb
        Color.FromArgb(230, 230, 250),   // Lavandă
        90f))                        // Unghi gradient: vertical
    {
        e.Graphics.FillRectangle(brush, this.ClientRectangle);
    }
}
```

Listing 9 – Gradient de fundal prin evenimentul Paint

3. Partea II – Aplicație vizuală complexă cu implementarea controllerelor frecvent utilizate

3.1. Componente frecvent utilizate și descrierea lor

3.1.1. Butoane de comandă (Button)

Aplicația conține 5 butoane principale create prin metoda helper CreateStyledButton(). Fiecare buton are o culoare semantică: verde pentru adăugare, albastru pentru modificare, roșu pentru ștergere, portocaliu pentru resetare, steel blue pentru filtrare.

```
// Creare butoane cu poziții și culori specifice
addButton    = CreateStyledButton("\u2795 Adaug\u0103", 40,  360, Color.LimeGreen);
editButton   = CreateStyledButton("\u270f\ufe0f Modific\u0103", 140, 360, Color.DodgerBlue);
deleteButton = CreateStyledButton("\U0001f5d1\ufe0f \u0218terge", 240, 360, Color.Crimson);
clearButton  = CreateStyledButton("\U0001f9fd Cur\u0103\u021b\u0103", 340, 360, Color.Orange);

// Legare evenimente Click la metodele handler
addButton.Click    += AddButton_Click;
editButton.Click   += EditButton_Click;
deleteButton.Click += DeleteButton_Click;
clearButton.Click  += (s, e) => ClearInputFields(); // Lambda inline
```

Listing 10 – Crearea și configurarea butoanelor principale

3.1.2. Casete de editare (TextBox)

Aplicația folosește 3 instanțe de TextBox cu scopuri diferite. nameTextBox și salaryTextBox sunt plasate în TableLayoutPanel, iar searchTextBox este plasat separat în zona de căutare:

```
// TextBox pentru Nume – plasat în TableLayoutPanel
nameTextBox = new TextBox()
{
   Font = new Font("Segoe UI", 10),
   BorderStyle = BorderStyle.FixedSingle,
   Anchor = AnchorStyles.Left | AnchorStyles.Right // Se întinde pe lățimea coloanei
};

// TextBox pentru Salariu – același stil
salaryTextBox = new TextBox()
{
   Font = new Font("Segoe UI", 10),
   BorderStyle = BorderStyle.FixedSingle,
   Anchor = AnchorStyles.Left | AnchorStyles.Right
};
```

```
// TextBox pentru Căutare – plasat separat, cu dimensiuni fixe
searchTextBox = new TextBox()
{
   Location = new Point(535, 458),
   Size = new Size(200, 28),
   Font = new Font("Segoe UI", 10),
   BorderStyle = BorderStyle.FixedSingle
};
```

Listing 11 – Configurarea instanțelor TextBox

3.1.3. Etichete (Label)

Etichetele sunt plasate în coloana stângă a TableLayoutPanel-ului, cu aliniere la dreapta pentru a se alinia vizual cu câmpurile din coloana dreaptă:

```
nameLabel = new Label()
{
   Text = "\U0001f464 Nume:",
   Font = new Font("Segoe UI", 10, FontStyle.Bold),
   TextAlign = ContentAlignment.MiddleRight, // Aliniere la dreapta
   AutoSize = true
};
roleLabel   = new Label() { Text = "\U0001f4bc Rol:", ... };
salaryLabel = new Label() { Text = "\U0001f4b0 Salariu (MDL):", ... };
descLabel   = new Label() { Text = "\U0001f4dd Descriere:", ... };

// Adăugare în TableLayoutPanel (coloana 0)
tableLayout.Controls.Add(nameLabel,   0, 0); // coloana 0, rândul 0
tableLayout.Controls.Add(roleLabel,   0, 1); // coloana 0, rândul 1
tableLayout.Controls.Add(salaryLabel, 0, 2); // coloana 0, rândul 2
tableLayout.Controls.Add(descLabel,   0, 3); // coloana 0, rândul 3
```

Listing 12 – Configurarea etichetelor în TableLayoutPanel

3.1.4. Cutii de editare multilinie (RichTextBox)

RichTextBox oferă suport pentru text formatat (bold, italic, culori) spre deosebire de TextBox simplu. În aplicație este utilizat pentru câmpul Descriere:

```
descriptionRichTextBox = new RichTextBox()
{
   Font = new Font("Segoe UI", 10),
   BorderStyle = BorderStyle.FixedSingle,
   Height = 60,                    // Înălțime fixă pentru 2-3 rânduri
   Anchor = AnchorStyles.Left | AnchorStyles.Right
};

// Citire conținut la adăugare/modificare:
Description = descriptionRichTextBox.Text.Trim()
```

```
// Resetare conținut:
descriptionRichTextBox.Clear(); // Metodă Clear() din RichTextBox
```

Listing 13 – Configurarea și utilizarea RichTextBox

3.1.5. Meniuri (MenuStrip, ToolStripMenuItem, ToolStripSeparator)

Bara de meniu este inițializată în metoda InitializeMenu(). Structura ierarhică a meniurilor este construită prin adăugarea ToolStripMenuItem în colecția DropDownItems a meniului părinte. Un ToolStripSeparator adaugă o linie de separare vizuală:

```
// Structura meniului Fișier:
fileMenu.DropDownItems.Add(exportMenuItem);     // Exportă în CSV
fileMenu.DropDownItems.Add(new ToolStripSeparator()); // Linie separatoare
fileMenu.DropDownItems.Add(exitMenuItem);       // Ieșire

// Structura meniului Editare:
editMenu.DropDownItems.Add(clearMenuItem);      // Curăță câmpuri

// Structura meniului Ajutor:
helpMenu.DropDownItems.Add(aboutMenuItem);      // Despre

// Adăugare meniuri în bara principală:
mainMenu.Items.Add(fileMenu);
mainMenu.Items.Add(editMenu);
mainMenu.Items.Add(helpMenu);
```

Listing 14 – Structura ierarhică a meniului

3.1.6. Grupuri de butoane (RadioButton în Panel)

RadioButton-urile sunt plasate într-un Panel pentru a asigura gruparea lor și selecția mutuală exclusivă. Dacă ar fi plasate direct pe formular, toate RadioButton-urile de pe formular ar forma un singur grup, ceea ce ar putea cauza conflicte:

```
rolePanel = new Panel() { Height = 40, AutoSize = true };

developerRadio = new RadioButton()
{
   Text     = "\U0001f468\u200d\U0001f4bb Developer",
   AutoSize = true,
   Location = new Point(0, 5)   // Prima poziție în panel
};
testerRadio = new RadioButton()
{
   Text     = "\U0001f9ea Tester",
   AutoSize = true,
   Location = new Point(110, 5) // A doua poziție
};
managerRadio = new RadioButton()
```

```
{
   Text     = "\U0001f4ca Manager",
   AutoSize = true,
   Location = new Point(220, 5) // A treia poziție
};

// Adăugare în panel – creează grupul de selecție exclusivă
rolePanel.Controls.AddRange(
   new Control[] { developerRadio, testerRadio, managerRadio });

// Adăugare panel în TableLayoutPanel (coloana 1, rândul 1)
tableLayout.Controls.Add(roleLabel, 0, 1);
tableLayout.Controls.Add(rolePanel, 1, 1);
```

Listing 15 – RadioButton-uri grupate în Panel

3.1.7. Cutii de grupare (GroupBox)

GroupBox oferă o delimitare vizuală clară a câmpurilor de introducere a datelor angajatului, cu un titlu afișat în cadrul bordurii:

```
employeeGroupBox = new GroupBox()
{
   Text     = "\U0001f4cb Date angajat", // Titlul afișat în bordură
   Location = new Point(25, 45),
   Size     = new Size(420, 300),
   Font     = new Font("Segoe UI", 11, FontStyle.Bold),
   ForeColor = Color.DarkSlateBlue,     // Culoare titlu
   Padding  = new Padding(10)           // Spațiu interior
};

// TableLayoutPanel este adăugat în GroupBox cu Dock.Fill
tableLayout.Dock = DockStyle.Fill;
employeeGroupBox.Controls.Add(tableLayout);
```

Listing 16 – Configurarea GroupBox

3.2. Adăugarea, excluderea și modificarea ferestrelor (MDI)

Form1 este configurat ca container MDI prin proprietatea IsMdiContainer = true. La încărcarea formularului (evenimentul Load), se creează și afișează automat un formular copil MDI cu informații despre aplicație:

```
protected override void OnLoad(EventArgs e)
{
   base.OnLoad(e);

   // Creare fereastră MDI copil
   Form child = new Form();
   child.Text    = "\U0001f31f Informa\u021bii companie";
```

```
    child.Size    = new Size(350, 200);
    child.MdiParent = this;            // Setare container MDI părinte
    child.BackColor = Color.FromArgb(240, 248, 255);

    // Adăugare conținut în fereastra copil
    Label info = new Label()
    {
      Text = "Aplica\u021bie modern\u0103 pentru gestionarea angaja\u021bilor.\n"
          + "MDI, controllere de baz\u0103, design atractiv.\n"
          + "Func\u021bii: c\u0103utare, statistici, export CSV.\n"
          + "Creat pentru studiul individual.",
      Dock      = DockStyle.Fill,
      TextAlign = ContentAlignment.MiddleCenter,
      Font      = new Font("Segoe UI", 10),
      ForeColor = Color.DarkSlateBlue
    };
    child.Controls.Add(info);

    child.Show(); // Metodă: afișare fereastră MDI copil
  }
```

Listing 17 – Crearea și afișarea ferestrei MDI copil

Pentru a adăuga mai multe ferestre MDI copil (ex. un formular de setări sau rapoarte), se repetă același tipar: new Form() → .MdiParent = this → .Show(). Ferestrele copil pot fi aranjate automat prin metode ca LayoutMdi(MdiLayout.TileHorizontal).

3.3. Modificarea stării și comportamentul componentelor conform specificațiilor

Comportamentul componentelor se modifică dinamic pe parcursul execuției. Principalele scenarii de modificare a stării sunt:

Scenariul 1: Selectare angajat din DataGridView

```
  // Când utilizatorul selectează un rând în DataGridView:
  private void DataGridView_SelectionChanged(object sender, EventArgs e)
  {
    if (employeesDataGridView.SelectedRows.Count > 0)
    {
      currentEmployee =
        (Employee)employeesDataGridView.SelectedRows[0].DataBoundItem;
      if (currentEmployee != null)
      {
        nameTextBox.Text = currentEmployee.Name;
        salaryTextBox.Text = currentEmployee.Salary.ToString();
        descriptionRichTextBox.Text = currentEmployee.Description;
```

```
            switch (currentEmployee.Role)
            {
               case "Developer": developerRadio.Checked = true; break;
               case "Tester":    testerRadio.Checked = true; break;
               case "Manager":   managerRadio.Checked = true; break;
            }
            editButton.Enabled   = true;  // Activare buton Modifică
            deleteButton.Enabled = true;  // Activare buton Șterge
         }
      }
      else { editButton.Enabled = false; deleteButton.Enabled = false; }
   }
```

Listing 18 – Modificarea stării la selectarea unui angajat

Scenariul 2: Resetarea câmpurilor

```
   private void ClearInputFields()
   {
      nameTextBox.Clear();            // Metodă Clear() pe TextBox
      salaryTextBox.Clear();
      descriptionRichTextBox.Clear();   // Metodă Clear() pe RichTextBox
      developerRadio.Checked = false;   // Proprietate Checked = false
      testerRadio.Checked    = false;
      managerRadio.Checked   = false;
      currentEmployee       = null;    // Resetare referință angajat curent
      editButton.Enabled     = false;   // Dezactivare butoane
      deleteButton.Enabled   = false;
   }
```

Listing 19 – Metoda ClearInputFields() pentru resetarea stării

3.4. Conversia șirurilor de caractere

Conversia și validarea datelor sunt esențiale pentru asigurarea integrității datelor introduse de utilizator. Metoda ValidateAndConvert() centralizează toată logica de validare:

```
   private bool ValidateAndConvert(out int salary)
   {
      salary = 0; // Parametru out inițializat cu valoarea implicită

      // Validare câmp Nume: nu poate fi gol sau doar spații
      if (string.IsNullOrWhiteSpace(nameTextBox.Text))
      {
         MessageBox.Show("Introduce\u021bi numele!", "Eroare",
            MessageBoxButtons.OK, MessageBoxIcon.Warning);
         return false;
      }

      // Validare câmp Salariu: nu poate fi gol
```

```
    if (string.IsNullOrWhiteSpace(salaryTextBox.Text))
    {
        MessageBox.Show("Introduce\u021bi salariul!", "Eroare",
            MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return false;
    }

    // Conversie string -> int cu validare
    // int.TryParse returnează false dacă șirul nu e număr valid
    if (!int.TryParse(salaryTextBox.Text, out salary) || salary <= 0)
    {
        MessageBox.Show(
            "Salariul trebuie s\u0103 fie un num\u0103r \u00eentreg pozitiv!",
            "Eroare", MessageBoxButtons.OK, MessageBoxIcon.Error);
        return false;
    }

    // Validare selecție rol
    string role = GetSelectedRole();
    if (string.IsNullOrEmpty(role))
    {
        MessageBox.Show("Selecta\u021bi un rol!", "Eroare",
            MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return false;
    }

    return true; // Toate validările au trecut cu succes
}
```

Listing 20 – Metoda ValidateAndConvert() cu conversii și validări

Conversia inversă (int -> string) se realizează prin metoda .ToString() la afișarea datelor:

```
// Conversie int -> string pentru afișare în TextBox
salaryTextBox.Text = currentEmployee.Salary.ToString();

// Conversie pentru afișare în StatusStrip (double -> string formatat)
double avgSalary = filteredEmployees.Average(e => e.Salary);
statusLabel.Text =
    $"\U0001f4ca Total angaja\u021bi: {count}  |  " +
    $"\U0001f4b0 Salariu mediu: {avgSalary:F2} MDL"; // F2 = 2 zecimale
```

Listing 21 – Conversii inverse int->string și double->string formatat

4. Partea III – Aplicație vizuală cu utilizarea controllerelor pentru lucrul cu bazele de date

4.1. Stabilirea conexiunilor cu bazele de date

În versiunea actuală, datele sunt gestionate în memorie printr-o colecție List<Employee>. Aceasta reprezintă un strat de date simplu, fără persistență între sesiuni. Datele de test sunt încărcate la pornire prin metoda LoadSampleData():

```
private List<Employee> employees = new List<Employee>();
private List<Employee> filteredEmployees = new List<Employee>();
private Employee currentEmployee; // Angajatul selectat curent

private void LoadSampleData()
{
   employees.Clear();
   employees.Add(new Employee
      { Name = "Ana Popescu",   Role = "Developer",
        Salary = 25000, Description = "Backend .NET" });
   employees.Add(new Employee
      { Name = "Ion Creang\u0103", Role = "Tester",
        Salary = 18000, Description = "Testare automat\u0103" });
   employees.Add(new Employee
      { Name = "Maria Ionescu", Role = "Manager",
        Salary = 32000, Description = "Conduce echipa" });
   filteredEmployees = new List<Employee>(employees); // Copie pentru filtrare
}
```

Listing 22 – Structura de date in-memory și datele de test

Pentru o extindere cu bază de date reală (ex. SQL Server sau SQLite), conexiunea s-ar stabili prin ADO.NET:

```
// Exemplu conexiune SQL Server (extensie viitoare):
string connectionString =
   "Server=localhost;Database=CompanyIT;Trusted_Connection=True;";
using (SqlConnection conn = new SqlConnection(connectionString))
{
   conn.Open(); // Deschidere conexiune
   // ... operații pe baza de date
}
```

Listing 23 – Exemplu conexiune ADO.NET (extensie viitoare)

4.2. Verificarea conexiunilor cu bazele de date

Verificarea și gestionarea excepțiilor la accesul datelor este demonstrată în aplicație prin metoda ExportToCsv(), care gestionează erorile la scrierea fișierului:

```
private void ExportToCsv(object sender, EventArgs e)
{
    using (SaveFileDialog sfd = new SaveFileDialog())
    {
        sfd.Filter   = "CSV files (*.csv)|*.csv";
        sfd.Title    = "Export\u0103 lista angaja\u021bilor";
        sfd.FileName = "angajati_export.csv";

        if (sfd.ShowDialog() == DialogResult.OK)
        {
            try // Bloc try-catch pentru gestionarea erorilor
            {
                var lines = new List<string> { "Nume;Rol;Salariu;Descriere" };
                lines.AddRange(filteredEmployees.Select(emp =>
                    $"{emp.Name};{emp.Role};{emp.Salary};{emp.Description}"));
                File.WriteAllLines(sfd.FileName, lines); // Scriere fișier
                MessageBox.Show(
                    $"Export reu\u015fit!\nFi\u015fier salvat la: {sfd.FileName}",
                    "Export CSV", MessageBoxButtons.OK,
                    MessageBoxIcon.Information);
            }
            catch (Exception ex) // Capturare eroare
            {
                MessageBox.Show($"Eroare la export: {ex.Message}",
                    "Eroare", MessageBoxButtons.OK, MessageBoxIcon.Error);
            }
        }
    }
}
```

Listing 24 – Export CSV cu gestionarea excepțiilor

4.3. Securizarea accesului la bazele de date

Securizarea accesului la date presupune mai multe niveluri de protecție. Aplicația actuală implementează validarea datelor de intrare ca prim nivel de securitate. Pentru o bază de date reală, se recomandă:

- Utilizarea parametrilor în interogările SQL pentru prevenirea SQL Injection – în loc de concatenare de șiruri.
- Stocarea connection string-ului în fișiere de configurare criptate (app.config cu secțiuni protejate).
- Implementarea unui sistem de autentificare cu roluri: administrator (CRUD complet) vs. utilizator standard (doar citire).
- Utilizarea principiului least privilege – contul de baze de date să aibă doar permisiunile strict necesare.

```
// Exemplu interogare parametrizată (securizată contra SQL Injection):
string query = "SELECT * FROM Employees WHERE Name = @name AND Role = @role";
using (SqlCommand cmd = new SqlCommand(query, conn))
{
   cmd.Parameters.AddWithValue("@name", nameTextBox.Text.Trim());
   cmd.Parameters.AddWithValue("@role", GetSelectedRole());
   // @name și @role sunt tratate ca date, nu ca cod SQL
}
```

Listing 25 – Exemplu interogare parametrizată (prevenire SQL Injection)

4.4. Implementarea operațiilor CRUD

Aplicația implementează complet cele 4 operații CRUD (Create, Read, Update, Delete) pe colecția in-memory:

CREATE – Adăugare angajat nou:

```
private void AddButton_Click(object sender, EventArgs e)
{
   if (ValidateAndConvert(out int salary)) // Validare și conversie
   {
      Employee emp = new Employee
      {
         Name       = nameTextBox.Text.Trim(),
         Role       = GetSelectedRole(),
         Salary     = salary,
         Description = descriptionRichTextBox.Text.Trim()
      };
      employees.Add(emp);       // CREATE: adăugare în colecție
      ApplyFilter();            // Reîmprospătare DataGridView
      ClearInputFields();       // Resetare câmpuri
      MessageBox.Show("\u2705 Angajat ad\u0103ugat cu succes!",
         "Informa\u021bie", MessageBoxButtons.OK,
         MessageBoxIcon.Information);
   }
}
```

Listing 26 – Operația CREATE (AddButton_Click)

READ – Afișare și filtrare angajați:

```
private void ApplyFilter()
{
   string searchTerm = searchTextBox.Text.Trim().ToLower();
   if (string.IsNullOrEmpty(searchTerm))
      filteredEmployees = new List<Employee>(employees); // READ: toți
```

```
    else
        // READ cu filtru LINQ: selecție după nume sau rol
        filteredEmployees = employees.Where(emp =>
            emp.Name.ToLower().Contains(searchTerm) ||
            emp.Role.ToLower().Contains(searchTerm)).ToList();

    RefreshDataGridView(); // Actualizare grilă
}

private void RefreshDataGridView()
{
    employeesDataGridView.DataSource = null;
    employeesDataGridView.DataSource = filteredEmployees; // Binding date
    employeesDataGridView.AutoResizeColumns();          // Ajustare coloane
    UpdateStatistics();                                 // Actualizare statistici
}
```

Listing 27 – Operația READ cu filtrare LINQ

UPDATE – Modificare angajat existent:

```
private void EditButton_Click(object sender, EventArgs e)
{
    if (currentEmployee == null)
    {
        MessageBox.Show(
            "Selecta\u021bi un angajat din list\u0103 pentru modificare!",
            "Avertisment", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }
    if (ValidateAndConvert(out int salary))
    {
        // UPDATE: modificare proprietăților obiectului referit
        currentEmployee.Name        = nameTextBox.Text.Trim();
        currentEmployee.Role        = GetSelectedRole();
        currentEmployee.Salary      = salary;
        currentEmployee.Description = descriptionRichTextBox.Text.Trim();
        ApplyFilter();   // Reîmprospătare (obiectul e același în colecție)
        ClearInputFields();
        MessageBox.Show("\u270f\ufe0f Date modificate corect!",
            "Succes", MessageBoxButtons.OK, MessageBoxIcon.Information);
    }
}
```

Listing 28 – Operația UPDATE (EditButton_Click)

DELETE – Ștergere angajat:

```
private void DeleteButton_Click(object sender, EventArgs e)
{
```

```
    if (currentEmployee == null)
    {
        MessageBox.Show(
            "Selecta\u021bi un angajat din list\u0103 pentru \u015ftergere!",
            "Avertisment", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }
    // Confirmare înainte de ștergere – bună practică UX
    if (MessageBox.Show(
        $"Sigur dori\u021bi s\u0103 \u015fterge\u021bi angajatul {currentEmployee.Name}?",
        "Confirmare", MessageBoxButtons.YesNo,
        MessageBoxIcon.Question) == DialogResult.Yes)
    {
        employees.Remove(currentEmployee); // DELETE: eliminare din colecție
        ApplyFilter();
        ClearInputFields();
        MessageBox.Show("\U0001f5d1\ufe0f Angajat \u015fters.",
            "Informa\u021bie", MessageBoxButtons.OK,
            MessageBoxIcon.Information);
    }
}
```

Listing 29 – Operația DELETE (DeleteButton_Click)

4.5. Implementarea interogărilor de selecție, modificare, inserare și excludere

Interogările sunt implementate prin operații LINQ pe colecția in-memory, echivalente cu instrucțiunile SQL SELECT, UPDATE, INSERT, DELETE:

```
// SELECT cu WHERE – filtrare după nume sau rol:
filteredEmployees = employees
    .Where(emp => emp.Name.ToLower().Contains(searchTerm)
            || emp.Role.ToLower().Contains(searchTerm))
    .ToList();

// INSERT – echivalent cu employees.Add(emp) din AddButton_Click
employees.Add(new Employee { Name = "...", Role = "...", ... });

// UPDATE – modificare directă a proprietăților obiectului referit:
currentEmployee.Name   = nameTextBox.Text.Trim();
currentEmployee.Salary = salary;

// DELETE – eliminare din colecție:
employees.Remove(currentEmployee);

// SELECT cu agregare – statistici:
int count      = filteredEmployees.Count;
double avgSalary = count > 0
    ? filteredEmployees.Average(e => e.Salary)
    : 0;
```

Listing 30 – Interogări LINQ echivalente operațiilor SQL

4.6. Crearea și afișarea rapoartelor

Raportarea statistică este implementată prin metoda UpdateStatistics(), care calculează și afișează în StatusStrip numărul total de angajați și salariul mediu al listei filtrate:

```
private void UpdateStatistics()
{
    int count = filteredEmployees.Count;
    double avgSalary = count > 0
        ? filteredEmployees.Average(e => e.Salary)
        : 0;
    // Afișare raport statistic în StatusStrip
    statusLabel.Text =
        $"\U0001f4ca Total angaja\u021bi: {count}  |  " +
        $"\U0001f4b0 Salariu mediu: {avgSalary:F2} MDL";
}
```

Listing 31 – Metoda UpdateStatistics() pentru raportare

Raportul tabelar (export CSV) generează un fișier cu antet și câte o linie pentru fiecare angajat din lista filtrată:

```
// Generare raport CSV:
var lines = new List<string> { "Nume;Rol;Salariu;Descriere" }; // Antet
lines.AddRange(filteredEmployees.Select(emp =>
    $"{emp.Name};{emp.Role};{emp.Salary};{emp.Description}")); // Date
File.WriteAllLines(sfd.FileName, lines, System.Text.Encoding.UTF8);
```

Listing 32 – Generarea raportului CSV

5. Concluzie

Lucrarea de studiu individual a permis aprofundarea cunoștințelor privind programarea vizuală în C# cu Windows Forms, prin realizarea practică a unei aplicații funcționale și complete pentru gestionarea angajaților dintr-o companie IT. Toate obiectivele propuse au fost atinse: proiectarea interfeței, implementarea logicii de business, validarea datelor, și funcționalitățile avansate.

Cercetarea a demonstrat complexitatea reală a procesului de proiectare a interfeței utilizator, evidențiind importanța alegerii corecte a componentelor și a organizării ierarhice a acestora. Dificultățile principale întâmpinate au inclus: (1) alinierea corectă a componentelor prin TableLayoutPanel cu coloane de lățimi mixte (absolută + procentuală); (2) stilizarea avansată a butoanelor cu colțuri rotunjite prin GraphicsPath; (3) sincronizarea stării between controale (activare/dezactivare butoane în funcție de selecție).

Posibilele direcții viitoare de dezvoltare, pornind de la limitele actuale ale aplicației, includ: (1) integrarea cu o bază de date reală SQL Server sau SQLite prin Entity Framework, pentru persistența datelor între sesiuni; (2) implementarea unui sistem de autentificare cu roluri (admin/utilizator); (3) adăugarea de grafice statistice interactive (Chart control sau LiveCharts); (4) export rapoarte în format PDF prin ReportViewer sau iTextSharp; (5) implementarea funcționalității de Undo/Redo pentru operațiile CRUD.

Personal, apreciez că această lucrare a oferit o perspectivă completă și practică asupra ciclului de viață al dezvoltării unei aplicații desktop: de la analiza cerințelor și proiectarea interfeței, până la implementarea logicii de business, validarea datelor și livrarea funcționalităților cerute. Aplicația CompanyITApp constituie o bază solidă și bine structurată pentru extinderi viitoare.

6. Bibliografie

1. Microsoft Corporation. C# Programming Guide. Microsoft Docs, 2024. https://learn.microsoft.com/en-us/dotnet/csharp/
2. Microsoft Corporation. Windows Forms documentation for .NET. Microsoft Docs, 2024. https://learn.microsoft.com/en-us/dotnet/desktop/winforms/
3. Microsoft Corporation. DataGridView Control – Windows Forms. Microsoft Docs, 2024. https://learn.microsoft.com/en-us/dotnet/desktop/winforms/controls/datagridview-control-overview-windows-forms
4. Microsoft Corporation. MenuStrip Control Overview. https://learn.microsoft.com/en-us/dotnet/desktop/winforms/controls/menustrip-control-overview-windows-forms
5. Price, M. J. C# 12 and .NET 8 – Modern Cross-Platform Development Fundamentals. Packt Publishing, Birmingham, 2023.
6. Albahari, J., Albahari, B. C# 12 in a Nutshell: The Definitive Reference. O'Reilly Media, Sebastopol, 2024.
7. Troelsen, A., Japikse, P. Pro C# 10 with .NET 6. Apress, New York, 2022.

7. Anexe

Anexa 1 – Codul sursă complet al aplicației CompanyITApp

Mai jos este prezentat codul sursă integral al aplicației, organizat pe secțiuni:

Declarații câmpuri și clasa Employee:

```
namespace CompanyITApp
{
   public partial class Form1 : Form
   {
      // Controale UI
      private MenuStrip mainMenu;
      private ToolStripMenuItem fileMenu, editMenu, helpMenu;
      private ToolStripMenuItem exitMenuItem, clearMenuItem,
                        aboutMenuItem, exportMenuItem;
      private GroupBox employeeGroupBox;
      private TableLayoutPanel tableLayout;
      private Label nameLabel, roleLabel, salaryLabel, descLabel;
      private TextBox nameTextBox, salaryTextBox;
      private RichTextBox descriptionRichTextBox;
      private Panel rolePanel;
      private RadioButton developerRadio, testerRadio, managerRadio;
      private Button addButton, editButton, deleteButton, clearButton;
      private DataGridView employeesDataGridView;
      private TextBox searchTextBox;
      private Button searchButton;
      private StatusStrip statusStrip;
      private ToolStripStatusLabel statusLabel;

      private List<Employee> employees = new List<Employee>();
      private List<Employee> filteredEmployees = new List<Employee>();
      private Employee currentEmployee;

      // Clasa model pentru angajat
      public class Employee
      {
         public string Name       { get; set; }
         public string Role       { get; set; }
         public int    Salary     { get; set; }
         public string Description { get; set; }
      }
```

Listing 33 – Declarații câmpuri și clasa Employee

Constructorul și evenimentul Paint:

```
public Form1()
{
   this.Text = "\u2728 Companie IT - Gestionare Angaja\u021bi \u2728";
   this.Size = new Size(1050, 700);
   this.StartPosition = FormStartPosition.CenterScreen;
   this.IsMdiContainer = true;
   this.BackColor = Color.White;
   this.Paint += Form1_Paint;
   InitializeMenu();
   InitializeControls();
   LoadSampleData();
   ApplyFilter();
   UpdateStatistics();
}

private void Form1_Paint(object sender, PaintEventArgs e)
{
   using (LinearGradientBrush brush = new LinearGradientBrush(
      this.ClientRectangle,
      Color.FromArgb(240, 248, 255),
      Color.FromArgb(230, 230, 250), 90f))
   {
      e.Graphics.FillRectangle(brush, this.ClientRectangle);
   }
}
```

Listing 34 – Constructorul Form1 și evenimentul Paint

Metodele CRUD și ajutătoare:

```
private string GetSelectedRole()
{
   if (developerRadio.Checked) return "Developer";
   if (testerRadio.Checked)   return "Tester";
   if (managerRadio.Checked)  return "Manager";
   return "";
}

private bool ValidateAndConvert(out int salary)
{
   salary = 0;
   if (string.IsNullOrWhiteSpace(nameTextBox.Text))
   { MessageBox.Show("Introduce\u021bi numele!", "Eroare",
     MessageBoxButtons.OK, MessageBoxIcon.Warning); return false; }
   if (string.IsNullOrWhiteSpace(salaryTextBox.Text))
   { MessageBox.Show("Introduce\u021bi salariul!", "Eroare",
     MessageBoxButtons.OK, MessageBoxIcon.Warning); return false; }
   if (!int.TryParse(salaryTextBox.Text, out salary) || salary <= 0)
   { MessageBox.Show("Salariul trebuie s\u0103 fie un num\u0103r " +
     "\u00eentreg pozitiv!", "Eroare",
```

```
    MessageBoxButtons.OK, MessageBoxIcon.Error); return false; }
  if (string.IsNullOrEmpty(GetSelectedRole()))
  { MessageBox.Show("Selecta\u021bi un rol!", "Eroare",
    MessageBoxButtons.OK, MessageBoxIcon.Warning); return false; }
  return true;
}

private void AddButton_Click(object sender, EventArgs e)
{
  if (ValidateAndConvert(out int salary))
  {
    employees.Add(new Employee {
      Name        = nameTextBox.Text.Trim(),
      Role        = GetSelectedRole(),
      Salary      = salary,
      Description = descriptionRichTextBox.Text.Trim() });
    ApplyFilter(); ClearInputFields();
    MessageBox.Show("\u2705 Angajat ad\u0103ugat cu succes!",
      "Informa\u021bie", MessageBoxButtons.OK,
      MessageBoxIcon.Information);
  }
}

private void EditButton_Click(object sender, EventArgs e)
{
  if (currentEmployee == null) {
    MessageBox.Show(
      "Selecta\u021bi un angajat pentru modificare!",
      "Avertisment", MessageBoxButtons.OK,
      MessageBoxIcon.Warning); return; }
  if (ValidateAndConvert(out int salary))
  {
    currentEmployee.Name        = nameTextBox.Text.Trim();
    currentEmployee.Role        = GetSelectedRole();
    currentEmployee.Salary      = salary;
    currentEmployee.Description = descriptionRichTextBox.Text.Trim();
    ApplyFilter(); ClearInputFields();
    MessageBox.Show("\u270f\ufe0f Date modificate corect!",
      "Succes", MessageBoxButtons.OK, MessageBoxIcon.Information);
  }
}

private void DeleteButton_Click(object sender, EventArgs e)
{
  if (currentEmployee == null) {
    MessageBox.Show(
      "Selecta\u021bi un angajat pentru \u015ftergere!",
      "Avertisment", MessageBoxButtons.OK,
      MessageBoxIcon.Warning); return; }
  if (MessageBox.Show(
    $"Sigur dori\u021bi s\u0103 \u015fterge\u021bi {currentEmployee.Name}?",
```

```
            "Confirmare", MessageBoxButtons.YesNo,
            MessageBoxIcon.Question) == DialogResult.Yes)
        {
            employees.Remove(currentEmployee);
            ApplyFilter(); ClearInputFields();
            MessageBox.Show("\U0001f5d1\ufe0f Angajat \u015fters.",
                "Informa\u021bie", MessageBoxButtons.OK,
                MessageBoxIcon.Information);
        }
    }

    private void ApplyFilter()
    {
        string searchTerm = searchTextBox.Text.Trim().ToLower();
        if (string.IsNullOrEmpty(searchTerm))
            filteredEmployees = new List<Employee>(employees);
        else
            filteredEmployees = employees.Where(emp =>
                emp.Name.ToLower().Contains(searchTerm) ||
                emp.Role.ToLower().Contains(searchTerm)).ToList();
        RefreshDataGridView();
    }

    private void UpdateStatistics()
    {
        int count = filteredEmployees.Count;
        double avg = count > 0 ? filteredEmployees.Average(e => e.Salary) : 0;
        statusLabel.Text =
            $"\U0001f4ca Total: {count}  |  " +
            $"\U0001f4b0 Salariu mediu: {avg:F2} MDL";
    }
  }
}
```

Listing 35 – Codul sursă complet al metodelor CRUD și ajutătoare